

Module 5 Software Assurance and Secure SQL Report

Modern Software Concepts in Python

Lynn Wall

October 6, 2026

Final Check	Result
Pylint	10.00/10
Pytest	104 tests passed
Coverage	100% - 700 statements, 0 missed
Dependency graph	dependency.svg created successfully
Snyk	Required dependency scan completed
GitHub Actions	Module 5 CI completed successfully

1. Setting Up the Project with pip and uv

For this module, I kept the project setup reproducible by using a virtual environment and the requirements.txt file. The requirements file includes the packages the application needs, along with Pylint and pydeps for the Module 5 assurance work. I also added setup.py and tested the project with an editable install. This gives someone starting with a fresh copy of the project a clear way to install the same dependencies instead of trying to recreate my environment manually.

```
pip setup:
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 -m pip install -e .
```

```
uv setup:
uv venv
source .venv/bin/activate
uv pip sync requirements.txt
uv pip install -e .
```

2. Protecting the SQL from Injection

The main SQL security change was making sure user input is never inserted directly into a SQL string. In query_data.py, I used Psycopg's SQL tools to build the statement safely. A dynamic column name is handled with sql.Identifier(), while the search value and limit are sent separately as parameters to cursor.execute(). This means a value that looks like SQL is still treated as data instead of becoming part of the command. I also added a test using a malicious-looking search value to make sure it does not return all of the records or change the query's intended behavior.

```
statement = psycopg_sql.SQL("""
    SELECT *
    FROM applicants
```

```

WHERE {} = %s
LIMIT %s;
""").format(psycopg_sql.Identifier(column_name))

parameters = (search_value, safe_limit)
cursor.execute(statement, parameters)

```

3. Limiting Query Results

I added limits so the application cannot request an unlimited number of rows from the database. The secure search function uses `clamp_limit()` to keep the requested limit between 1 and 100. The aggregate queries that are expected to return one result use `LIMIT 1`. This keeps the queries bounded and follows the assignment requirement to include an inherent limit instead of relying on the user to provide a safe value.

4. Database Credentials and Least Privilege

I removed the need to hard-code database credentials in the Python source. The database settings now come from `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`. I created `.env.example` to show which values are required without putting my real password in the repository, and `.env` is ignored by Git.

I also created a separate PostgreSQL role named `gradcafe_app` for application access. I did not make this role a superuser or database owner. It was given `CONNECT` to the database, `USAGE` on the public schema, and `SELECT`, `INSERT`, and `UPDATE` on the application tables. I kept administrative permissions such as `DROP`, `ALTER`, `CREATEDB`, and `CREATEROLE` away from the application account. This way, the application has the access it needs without having full control of the database.

```

CREATE ROLE gradcafe_app WITH LOGIN PASSWORD '<stored securely>'
    NOSUPERUSER NOCREATEDB NOCREATEROLE;

GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO gradcafe_app;

```

5. Dependency Graph

I generated `dependency.svg` with `pydeps` and `Graphviz` using `src/app.py` as the starting point. The graph gave me a visual view of how the Flask application connects to the other Python modules in the project. I could see the database-related code separated between the SQLAlchemy model/ORM files and the Psycopg query and loading code. The pull-data portion also connects to the LLM-hosting code that is used when applicant records are standardized. Seeing the modules this way made the project structure easier to review than looking at imports one file at a time. It also helps show where changes in one part of the application could affect another part. The completed graph is saved as `dependency.svg` in `module_5`.

6. Packaging the Application

I added `setup.py` and confirmed that `pip install -e .` works. The purpose of packaging here is to make the project installable and keep imports consistent instead of depending on the exact folder I happen to be working from. The editable install is useful while developing because I can change the source files without reinstalling the package after every edit. It also gives the local environment and GitHub Actions a more consistent way to work with the project.

7. Snyk Security Scan

I ran the required snyk test against the project dependencies. My local scan tested 62 dependencies and found one high-severity issue involving `diskcache 5.6.3`. `Diskcache` is not a dependency I added directly; it is brought in through `llama-cpp-python 0.3.35`. Snyk reported that there is no direct upgrade or patch available for this finding, so I documented the result rather than changing the course application's dependency to an untested version. The screenshot of the required scan is saved as `snyk-analysis.png`. I also attempted the optional snyk code test, but Snyk returned a 403 message stating that Snyk Code is not supported for my current organization, so I am not claiming the extra-credit SAST scan.

8. Pylint, Testing, and GitHub Actions

After working through the Pylint findings, the final `pylint src` run scored 10.00/10. I also ran the full test suite locally and finished with 104 passing tests and 100% coverage across 700 statements. The Module 5 GitHub Actions workflow runs on pushes and pull requests. It installs the project dependencies and `Graphviz`, checks Pylint with `--fail-under=10`, generates `dependency.svg`, runs Snyk, and runs Pytest. Because the Snyk finding has no direct upgrade or patch, the workflow records that scan but continues so the remaining required checks can run. The final Module 5 CI run completed successfully, and the green run is saved as `actions_success.png`.

9. Final Results

At the end of the module, the application still passed its existing tests while adding the security and assurance requirements for Module 5. The final results were Pylint 10.00/10, 104 passing tests, 100% coverage, a generated dependency graph, a completed Snyk dependency scan, and a successful Module 5 GitHub Actions run. The supporting files are included in the `module_5` folder with the source code and tests.